

Biblio-Bot

Documentation

Bienvenue sur la documentation de Biblio-Bot, le bot Discord qui vous permet de créer et gérer une bibliothèque de documents pour votre serveur. Vous retrouverez dans ce fichier :

- [Une présentation du fonctionnement du bot](#) ;
- [Des guides d'utilisation pour réaliser les principales actions](#) ;
- [Une référence \(en anglais\) des commandes](#).

Présentation

Biblio-Bot est, comme son nom l'indique, un bot Discord destiné à gérer une bibliothèque de documents pour votre serveur. Ces documents peuvent être de n'importe quel type (images, PDF, fichiers Word, musiques, ...) et sont stockés dans Google Drive, plus précisément dans un dossier de votre GDrive (ou de celui de votre serveur) que vous devrez partager au bot.

De manière basique, Biblio permet d'envoyer des fichiers dans le drive de manière à ce que n'importe qui du serveur puisse y accéder au travers du bot. Ainsi, il vous suffit d'autoriser le bot à accéder à votre dossier (et pas tous les utilisateurs du Discord) pour que tous les membres puissent récupérer et envoyer des documents. Cela permet d'éviter notamment des suppressions intempestives de fichier. De surcroît, la manière avec laquelle nous pouvons interagir avec le bot repose sur plusieurs principes fondamentaux qui permettent de réguler l'utilisation du bot.

➤ Distinction lecteurs/écrivains

Biblio fait la distinction entre les utilisateurs dits '*écrivain*' et les utilisateurs dits '*lecteurs*'. Cette différence entre les différents membres de votre serveur conditionne leur droit d'accès au drive :

- Les *lecteurs* peuvent seulement accéder aux fichiers présents sur le Drive de manière à les télécharger, ils n'ont aucune possibilité d'édition. Ils peuvent exécuter les commandes de lecture. Ces dernières ne peuvent être exécutées que dans des channels précis.
- Les *écrivains* peuvent envoyer des fichiers dans le Drive, les mettre à jour. Ils peuvent bien entendu supprimer les documents dont ils sont l'auteur et télécharger n'importe quel élément du Drive. Ils peuvent exécuter les commandes d'écriture (en plus de celles de lecture). Ces dernières ne peuvent être exécutées que dans des channels précis.

Pour réaliser cette distinction, Biblio se base sur le système de rôles propre à Discord. Ainsi, il est nécessaire de créer un ou plusieurs rôles pour les *écrivains* ainsi qu'un ou plusieurs rôles pour les *lecteurs*.

Il existe un troisième type d'utilisateur aux yeux du bot : les *administrateurs*. Il s'agit simplement des utilisateurs ayant les permissions administrateurs sur le serveur. Ces derniers ne peuvent exécuter que les commandes d'administration (réglages du bot).

➤ Domaines, niveaux et types

Pour trier les fichiers, le bot se base sur trois éléments : le **domaine** auquel appartient le document, le **niveau** auquel se destine le document et le **type**¹ de document dont il s'agit (par défaut le bot considère qu'il s'agit d'un cours). Ces éléments interfèrent dans le fonctionnement du bot :

- Chaque **domaine** que possède la bibliothèque est associé à une liste de channels ainsi qu'à un rôle. Cette liste de channels correspond aux channels dans lesquels les commandes d'écriture et de lecture pourront être exécutées (plus de détails sur les channels [ici](#)). Le rôle correspond au rôle des *écrivains* spécialisés dans ce **domaine**.
- Chaque **niveau** doit être relié à un rôle. Ce rôle correspond au rôle des *lecteurs* ayant ce **niveau**. (Il n'a pas non plus beaucoup d'impact sur le fonctionnement interne du bot).

Le fonctionnement du bot reposant en partie sur ces éléments, au moins un **domaine** et un **niveau** doivent être paramétrés. Cependant, pour simplifier l'accès au bot, plusieurs **domaine/niveau** peuvent avoir le(s) même(s) rôle/channel(s) et il est possible de définir en plus des channels qui ne sont reliés à aucune matière et qui permettent d'exécuter toutes les commandes (les *global channels*).

Attention si un **niveau** et un **domaine** pointe vers le même rôle, le bot ne pourra plus faire la différence entre les *écrivains* et les *lecteurs*.

➤ Système de validation

L'ensemble des documents sont, par défaut, soumis à un système de validation dit 'par les pairs'. Cela signifie que lorsqu'un *écrivain* poste un fichier, ce dernier n'est pas accessible aux *lecteurs* de manière immédiate. Il doit d'abord être validé par un autre *écrivain* qui est spécialisé dans le domaine que vise le document (un *écrivain* peut envoyer un document qui ne fait pas partie de ses domaines de compétence, cependant ce document doit être validé par quelqu'un possédant ces compétences).

À l'inverse, n'importe quel *écrivain* possédant le domaine d'un document peut le refuser même si ce dernier a déjà été validé. Cela a pour conséquence d'empêcher les *lecteurs* de le voir. Après un refus, un fichier ne peut plus être validé, le seul moyen de le débloquent est de faire une mise à jour du fichier qui sera de nouveau en attente de validation.

Le système de refus peut être désactivé (**les fichiers déjà refusés ne seront pas 'dé-refusés'**). Si l'ensemble du système de validation vous gêne vous pouvez le désactiver entièrement (**Cela empêche le refus et valide tout document envoyé automatiquement, et valide les fichiers non refusés. Les fichiers déjà refusés ne seront pas 'dé-refusés'**).

¹ N'a actuellement pratiquement aucun impact sur le fonctionnement interne du bot.

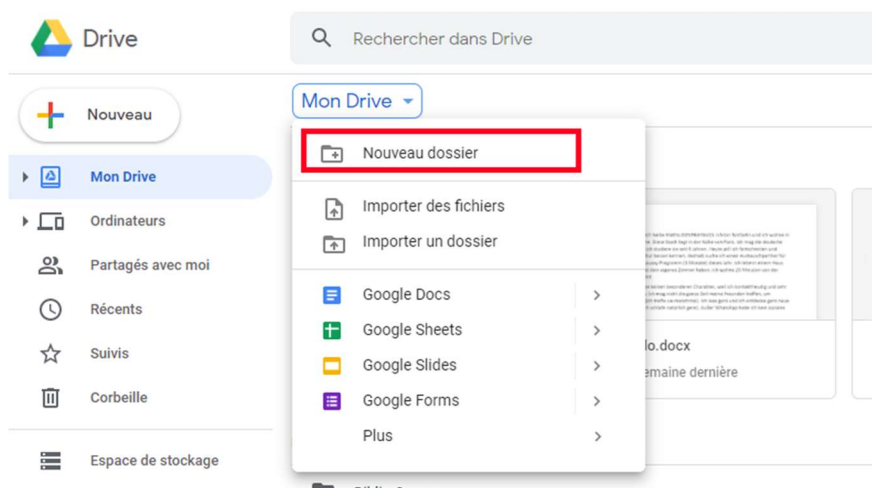
Guides

Les différents guides qui vont suivre ont pour but de vous aider à utiliser Biblio. Toutes les commandes doivent être envoyées sur le serveur. Précision pour les commandes d'écritures et de lecture, elles doivent être envoyées sur un channel lié à un **domaine** ou sur un *global channel*.

➤ Relier Biblio à votre GDrive

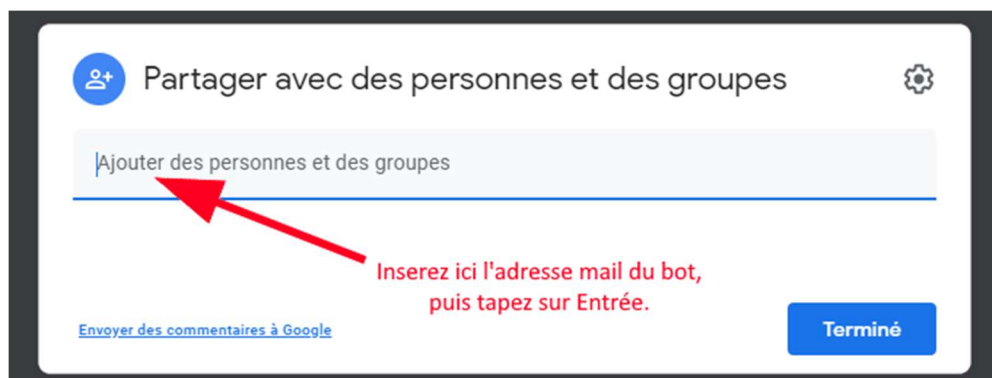
Ce guide (la mise en relation entre GDrive et le bot) ne doit être réalisé qu'une seule fois sur un serveur (faites attention actuellement aucune sécurité n'est implémentée). Il est donc uniquement à destination du propriétaire du serveur.

La première chose à faire est de choisir le GDrive que vous allez utiliser. Partez du principe que n'importe qui du Discord (et par extrapolation n'importe quelle personne de la planète) pourra accéder aux informations publiques du compte Google qui possède le GDrive. Ainsi je vous conseille de créer un compte pour votre serveur ne donnant aucune information sur vous (après vous faites comme vous voulez). Une fois votre GDrive choisi, vous devez y créer un dossier vierge :

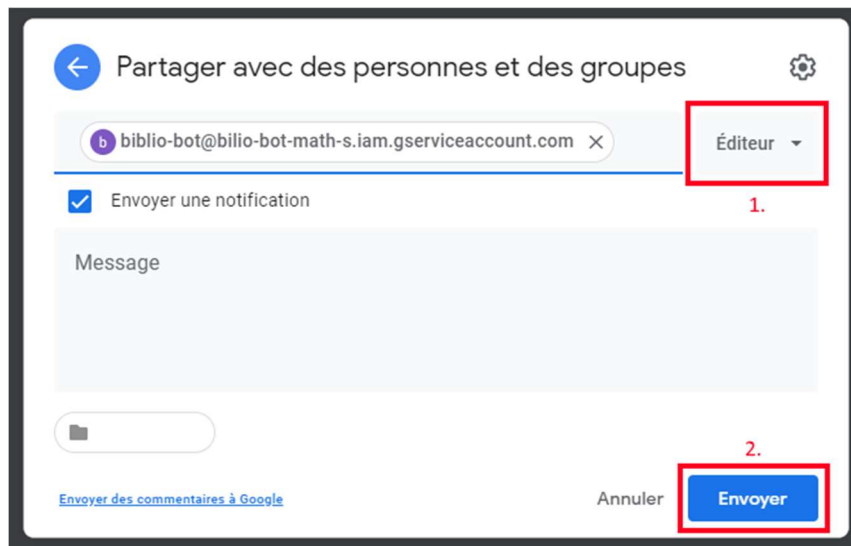


Une fois votre dossier créé, faites un clic-droit dessus puis appuyez sur 'Partager' dans le menu contextuel. Un champ de recherche s'affiche, rentrez-y l'adresse mail suivante (il s'agit de l'adresse mail du compte Google de Biblio), puis faites 'Entrée' :

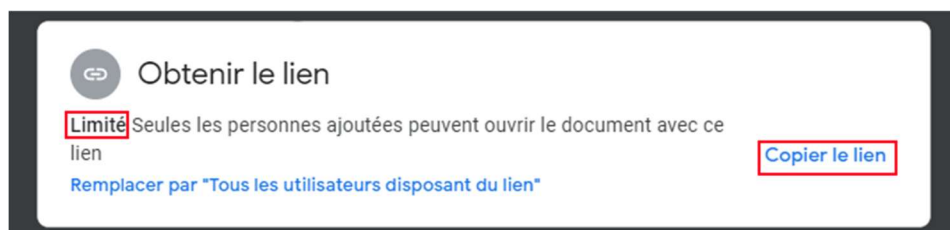
biblio-bot@bilio-bot-math-s.iam.gserviceaccount.com



Une nouvelle fenêtre de dialogue s'ouvre alors, faites alors bien attention à donner au bot les droits d'**Éditeur**. Vous pouvez alors appuyer sur envoyer (rien ne sert de remplir le champ de notifications, le bot ne l'analysera pas) :



Biblio dispose désormais des autorisations nécessaires pour accéder à votre dossier (je rappelle que ce dernier doit être vide et qu'aucune main humaine ne doit y toucher). Il cependant maintenant nécessaire de lui indiquer son URL. Pour cela faites de nouveaux un clic-droit sur votre dossier, puis appuyer sur 'Partager'. Il vous faut alors veiller à ce que le lien soit en mode 'Limité' ; vous pourrez ensuite le copier.



Il ne vous reste plus qu'à envoyer le lien au bot. Pour cela, placer dans n'importe quel salon de votre serveur Discord puis, après avoir ajouté le bot à votre serveur (en cliquant [ici](#)), envoyer la commande suivante :

```
Biblio: url -link: <The_copied_link>
```

Le bot est maintenant relié à votre GDrive. Il peut cependant arriver que cette commande ne voie pas l'URL et vous envoie le message 'URL is missing'. Vous pouvez dans ce cas l'exécuter de nouveaux.

➤ Créer des domaines et des niveaux

Pour faire fonctionner le bot, il faut maintenant lui indiquer au moins un **domaine** et un **niveau**. C'est ce que nous allons faire, et cela en seulement deux messages ! Ces deux messages utilisent la même commande : `create_settings`. Elle permet, comme son nom l'indique, de créer un paramètre. Pour signaler la création d'un **domaine**, il faut ajouter l'argument `-subject` ; pour un **niveau**, il s'agit de `-level`. Ainsi cela donne, pour le premier message :

```
Biblio: create_settings -subject:<domain_name> - #<A_channel> @<A_role>
```

Le bot va, grâce à ce message, créer un nouveau **domaine** du nom de `<domain_name>`. De plus tous les utilisateurs ayant le rôle `@<A_role>` pourront exécuter les commandes d'écritures (envoyer des fichiers, ...) car ils seront considérés comme des *écrivains* spécialisés dans ce domaine. Enfin, les commandes d'écritures et de lecture pourront être exécutées dans le channel `#<A_channel>`.

Pour créer un niveau, le message est plus court :

```
Biblio: create_settings - level:<level_name> - @<Another_role>
```

Cela créer un nouveau niveau dont le nom est : <domain_name>. De surcroît, tous les utilisateurs possédant le rôle @<Another_role> seront considérés comme des *lecteurs* possédant ce niveau.

➤ Gérer les channels de Biblio

Lors de la création d'un **domaine**, vous le liez à un channel. Tous ces channels seront les seuls dans lesquels vous pourrez communiquer avec le bot (sauf pour les commandes d'administration), il peut donc être utile d'en ajouter sans pour autant ajouter de **domaines**. Pour cela vous pouvez ajouter 2 types de channels :

- Un channel qui sera lié avec un **domaine** préexistant. Cela présente l'avantage de faciliter la recherche de documents dans le domaine en question ainsi que de limiter le flux dans ce channel (seuls les domaines associés à ce channel pourront être cherchés, le channel initialement lié se comporte de la même façon).
- Un channel dit 'global' où n'importe quoi peut être cherché mais qui ne facilite la recherche dans aucun channel.

La commande qui va suivre ne vous permet pas seulement d'ajouter un channel mais aussi de le retirer. Ainsi, si le channel dans la commande fait déjà parti de la liste, il sera retiré de cette dernière et s'il n'en fait pas encore parti, il y sera ajouté. Pour les channels de **domaine**, la commande est :

```
Biblio: modify_settings -subject:<Subject_Name> - #<Channel>
```

Et pour finir, voici la commande pour les channels dits 'généraux' :

```
Biblio: modify_settings -globalc - #<Channel>
```

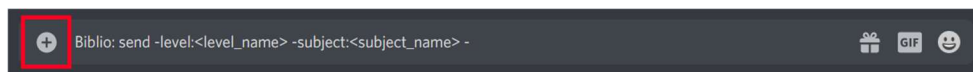
Vous pouvez désormais contrôler les channels dans lesquels les *lecteurs* et les *écrivains* peuvent interagir avec le bot et dans quelle mesure. Si vous ne voulez pas vous embêter avec ce système, je vous conseille d'ajouter un grand nombre de channel 'généraux' et de lier tous les **domaines** à un même channel.

➤ Envoyer un fichier et le valider

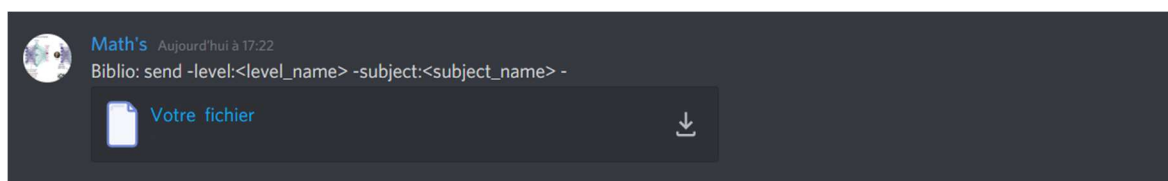
Envoyer un fichier est presque la plus simple des actions possibles. Cependant, il vous faut avant tout savoir quel **domaine** et quel **niveau** votre document vise, sans cela vous ne pourrez exécuter la commande. Pour envoyer votre fichier, le message est :

```
Biblio: send -level:<level_name> -subject:<subject_name> -
```

Attention ! N'envoyer pas de suite votre message, il faut aussi y mettre le contenu de votre fichier sous la forme d'un Attachment. Pour cela cliquer sur le petit + sur la gauche :



Un explorateur de fichier devrait alors s'ouvrir. Localisez et sélectionnez votre fichier puis cliquez sur 'Envoyer'. Une fois envoyé votre message doit contenir la commande et votre fichier, de la même manière que sur l'image :



Si le système de validation est activé, il vous faut maintenant faire valider votre document par un de vos pairs, ainsi la suite de ce guide s'adresse à toute personne désireuse de valider un document. Pour valider un fichier il faut connaître son ID. Pour le connaître il vous faut exécuter la commande suivante :

```
Biblio: search_unvalidate -subject:<A_subject>
```

Cette commande va vous retourner l'ensemble des documents en attente d'une validation (y compris les documents refusés) faisant partie du domaine <A_subject>. Dans la liste retournée sera notamment indiqué l'ID de chaque fichier. Une fois cet ID en main, il serait stupide de le valider de manière immédiate : il faut d'abord vérifier le contenu de ce fameux document. Pour cela, rien de plus simple :

```
Biblio: get_unvalidate -id:<An_ID>
```

Cette commande vous retourne le fichier choisi. Si vous êtes en accord avec le contenu, vous pouvez valider le document avec la commande :

```
Biblio: validate -id:<An_ID>
```

Le fichier est dorénavant validé et donc accessible à tous. Félicitations !

➤ Chercher et obtenir un document

Bien nous arrivons enfin à la partie où les fameux channels liés aux **domaines** sont utiles. En effet, nous allons commencer par rechercher des documents. Cela va nous permettre de choisir le fichier que l'on veut étudier mais aussi de trouver son ID. Pour chercher tous les documents qui possèdent un **type**, un **niveau** et un **domaine** donné, vous devez utiliser la commande :

```
Biblio: search -subject:<subject> -level:<level> [-type:<type>]
```

Cette commande va retourner l'ensemble des fichiers du serveur visant le domaine <subject>, le niveau <level> et le type <type> (ce dernier étant optionnel). Si vous vous trouvez dans un channel lié à différents **domaines**, le paramètre ne peut <subject> appartenir qu'à ces **domaines**. De plus si le channel est lié à un unique domaine, vous pouvez omettre le paramètre -subject. Le paramètre -level est aussi omissible mais dans ce cas, le bot cherchera à trouver votre niveau grâce à vos rôles.

Un fois votre document choisi et que vous avez récupéré son ID, il ne vous reste plus qu'à demander l'accès à son contenu au bot. Pour cela, rien de plus simple :

```
Biblio: get -id :<ID>
```

Vous devez normalement avoir obtenu, après quelques secondes d'attente, le contenu de votre fichier et vous pouvez l'utiliser normalement.

➤ Gérer les paramètres de votre Biblio

Il est possible de régler le fonctionnement de Biblio pour de manière à correspondre à vos besoins. Toutes les commandes qui vont suivre ne peuvent être exécuter que par des administrateurs. Vous pouvez tout d'abord désactiver le système de validation grâce à la commande suivante :

```
Biblio: modify_settings -validation:false
```

Cette commande va valider de manière immédiate tous les documents qui sont en attente d'une validation, sauf dans le cas des fichiers refusés. Par la suite, elle va empêcher le refus de cours et validera automatiquement n'importe contenu lors de son envoi (ainsi que lors d'une MAJ). Il vous est tjrs possible de le réactiver avec la commande :

```
Biblio: modify_settings -validation:true
```

Il est également possible de désactiver uniquement le système de refus :

```
Biblio: modify_settings -refuse:false
```

Cette commande empêche seulement les *écrivains* de refuser des fichiers. Pour réactiver le refus, vous devez envoyer le message suivant sur votre serveur :

```
Biblio: modify_settings -refuse:true
```

Au vu des modifications possibles des paramètres (il y a aussi les modifications de channels), il peut vous être utile de vérifier les paramètres qu'utilise actuellement Biblio. Pour cela il vous suffit de taper :

```
Biblio: handle_settings -view:run
```

Cette commande va vous renvoyer au format JSON les paramètres actuels. Attention, cela n'est pas forcément aisé à lire (il est de manière générale déconseillé d'utiliser la commande `handle_settings`).

Reference (English)

➤ Writers' commands

Here are the commands that can be executed by a *writer*:

- [send](#)
- [search_unvalidate](#)
- [get_unvalidate](#)
- [validate](#)
- [refuse](#)
- [update](#)
- [delete](#)

send -level:<level> -subject:<subject> [-type:<type>]

<your document>

Allows you to send a document to the bot (guide [here](#)).

-level: <level> => Indicates the document's level;

-subject: <subject> => Indicates the document's subject;

-type: <types> => Allow you to indicate the document's type (optional, by default a course);

-<your document> => Create an attachment to your document with discord.

Short version: sd

search_unvalidate -subject:<subject>

Allows you to see the documents that wait for a validation in the indicated subject (guide [here](#)).

-subject: <subject> => Indicate the documents' subject that you want to see;

Short version: su

get_unvalidate -id:<document's id>

Allows you to get the content of a file which wait for a validation or which has been refused (guide [here](#)).

-id: <document's id> => Indicate the document's ID.

Short version: gu

validate -id:<document's id>

Allows you to validate a document if it isn't already refused (guide [here](#)).

-id: <document's id> => Indicate the document's ID.

Short version: v

refuse -id:<document's id>

Allows you to refuse a document. A refused document can no more be validated.

-id: <document's id> => Indicate the document's ID.

Short version: r

update -id:<document's id>

Allows to update one of your documents.

-id: <document's id> => Indicate the document's ID.

Short version: up

delete -id:<document's id>

Allows to delete one of your documents.

-id: <document's id> => Indicate the document's ID.

Short version: del

➤ Readers' commands

Here are the commands that can be executed by a *reader*:

- [get](#)
- [search](#)

get -id:<document's id>

Allows you to get the content of a file (guide [here](#)).

-id: <document's id> => Indicate the document's ID.

Short version: g

search [-subject:<subject>] [-level:<level>]

Allows you to search all the validated document for a given subject and a given level (guide [here](#)).

-subject: <subject> => Indicate the documents' subject that you want to see. By default, the subject which is linked to the channel where the message is sent;

-level: <level> => Indicate the documents' subject that you want to see. By default, your level role.

Short version: s

➤ Administrators' commands

Here are the commands that can be executed by an *administrator*:

- [url](#)
- [create_settings](#)
- [modify_settings -subject](#)
- [modify_settings -level](#)
- [modify_settings -prof](#)
- [modify_settings -validation](#)
- [handle_settings](#)

url -link: <GDrive folder URL>

Allows you to bind your bot to a GDrive folder (one for each server).

-link: <GDrive folder URL> => Indicate the share URL of your folder.

```
create_settings [-subject:<subject_name>] [-  
level:<level_name>] - @<Role> [#<Channel>
```

Allows you to create a setting. The creatable settings are the subjects and the levels. You can't create two settings simultaneously. (more information [here](#) and [here](#)).

- subject: <subject_name>** => Use it just if you want to create a subject. Indicate the subject's name.
- level: <level_name>** => Use it just if you want to create a level. Indicate the level's name.
- **@<Role>** => Indicate an existing role, it will be bind to the setting.
- **#<Channel>** => Use it just if you want to create a subject. Indicate an existing Text Channel, it will be bind to the setting.

Short version: cs

```
modify_settings -subject:<subject_name> [-name:<new_name>  
- [@<Role>] [#<Channel>
```

Allows you to modify subject's name or bindings : channels and role (more information [here](#) and guide [here](#)).

- subject: <subject_name>** => Indicate the actual subject's name you want to modify.
- **name: <new_name>** => Indicate the new name of the subject.
- **@<Role>** => Indicate an existing role, it will be bind to the subject instead of the old one.
- **#<Channel>** => Indicate an existing TextChannel, it will be added to the channels' list of this subject if it's not already in the list, else it will be deleted from the list.

Short version: ms -subject

```
modify_settings -level:<level_name> [-name:<new_name> -  
[@<Role>]
```

Allows you to modify level's name or bindings : channels and role (more information [here](#)).

- subject: <level_name>** => Indicate the actual level's name you want to modify.
- name: <new_name>** => Indicate the new name of the level.
- **@<Role>** => Indicate an existing role, it will be bind to the level instead of the old one.

Short version: ms -level

```
modify_settings -globalc - #<Channel>
```

Allows modify *global channel's* list. (more information [here](#) and guide [here](#)).

- **#<Channel>** => Indicate an existing TextChannel, it will be added to the channels' list of this subject if it's not already in the list, else it will be deleted from the list.

Short version: ms -globalc

```
modify_settings -validation:<true|false>
```

Allows to enable or disable the validation system (more information [here](#) and guide [here](#)).

- validation: <true|false>** => Indicate 'true' (enable) or 'false'(disable).

Short version: ms -validation

modify_settings -refuse:<true|false>

Allows to enable or disable the refuse system (more information [here](#) and guide [here](#)).

-validation: <true|false> => Indicate 'true' (enable) or 'false' (disable).

Short version: ms -refuse

handle_settings [-view:<saved|run>] [-load] [-save]

Allows you to handle the guild's settings (guide [here](#)). It's not very useful, it was mainly for debugging.

-view: <saved|run> => Indicate 'saved' to get the settings saved on the disk and 'run' for the settings in the RAM. It's in JSON.

-load => Load the settings on the disk into RAM.

-save => Save the settings in the RAM onto the disk.

Short version: hs

prefix -<new_prefix>

Allows you to change the prefix of your bot. It's by default 'Biblio:' (yes, it's long)

-<new_prefix> => Indicate the new desired.

➤ DM commands

Here are the commands which can be sent to the bot through a DM by *anyone*. It allows you to define your settings. They don't need any prefix:

- [info](#)

info -<enable|disable>

Allows to choose if you the bot have to send you a message when one of your files is refused or validated (useful only for authors). By default they are enabled.

-<enable|disable> => Indicate 'disable' to disable these messages else indicate 'enable'.